

Unit Tests

Some aspects of the program can have their functionality ensured by rigorous unit testing. Because of the small number of internal programs and most of the system is contingent on user interaction, not much of the program can be unit tested.

Unit tests cover the Database Methods program as follows:

Test Number	Test Name	Intention	Outcome
1	testAddUser	Test the creation of users in the DB	Pass
2	testUserWeights	Test the initialisation and saving of user preferences in the DB	Pass
3	testAddPoints	Test the tracking of user points from missions in the DB	Pass
4	testDeleteUser	Test the deletion of a user from the DB	Pass
5	testGetNodes	Test the storage and initialisation of location nodes in the DB	Pass
6	testUpdateNode	Test the updating of location nodes in the DB	Pass
7	testGetAllEdges	Test the storage and initialisation of edges in the DB	Pass
8	testUpdateEdgeLength	Test the updating of edge lengths in the DB	Pass
9	testGetSurroundingLength	Test the foreign key functionality of nodes and edges in the DB	Pass
10	testDeleteNode	Test the deletion of nodes and ensure clean deletion which supports the foreign key constraints of the DB	Pass
11	testGetLocations	Test the saving, initialisation and	Pass

		collection of named locations from the DB	
12	testGetMissionData	Test the storage and retrieval of mission data in the DB	Pass
13	testEditMission	Test updating a mission in the DB	Pass

```
tcmor@laptop MINGW64 ~/Documents/Team Project/final pack/testing/testing
$ C:/Users/tcmor/anaconda3/python.exe "c:/Users/tcmor/Documents/Team Project/final pack/testing/testing/databaseTests.py"
.....
-----
Ran 13 tests in 0.526s

OK
```

Since all our unit tests pass we can be sure that the functionality of the individual components is robust and should not become problematic when we integrate them with user input. It was absolutely integral that the database handler passed all unit tests since the entire application requires reliable data at multiple stages.

To run the unit tests for yourself, navigate to the testing folder in the source code. Run [databaseTests.py](#) whilst in that folder.

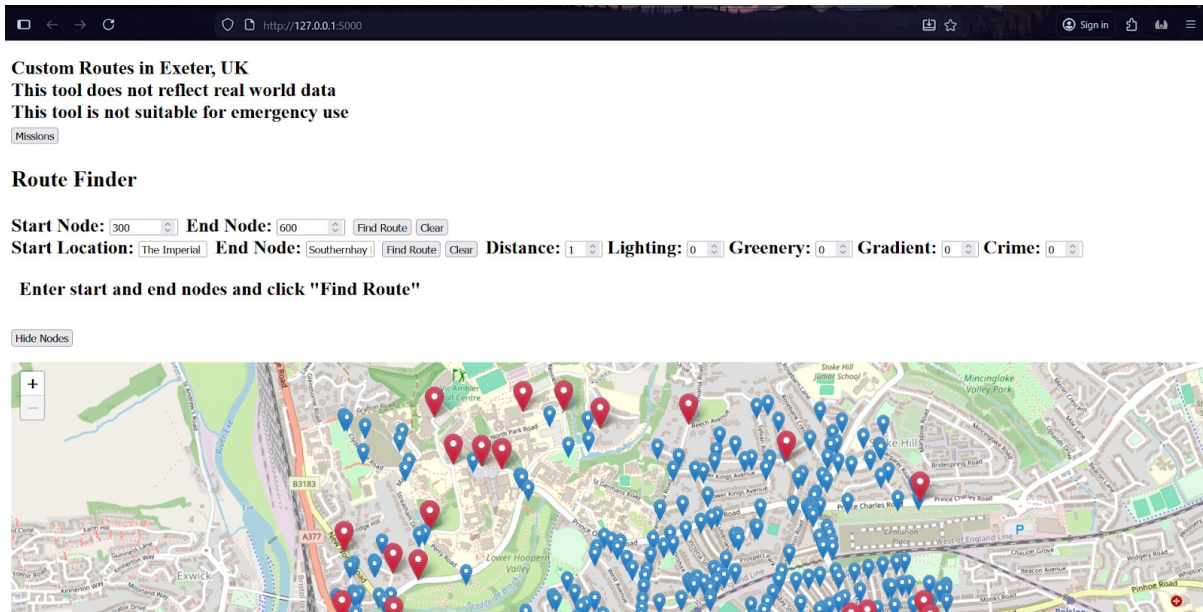
End to End Manual Tests

The manual testing for the project follows a similar structure to our demonstration in our presentation. We test the functionality of the user interface and how well it communicates with the backend of the server. To ensure good code coverage, interactions are comprehensively tested based on what URLs we serve to the user. Files covered by these tests are [server.py](#), [database_methods.py](#), [routefinding.py](#) and all of our html templates.

Since our security system is not fully implemented yet, by default the user is signed into an admin level account. Admin level accounts can still use the route-finding features as usual, as well as making changes. This is useful for the purpose of testing since we can see how admin changes affect route-finding and user experience.

Route-finding and Map Tests

First we start the server by running [server.py](#) and then navigate to <http://127.0.0.1:5000/> . We should now see the map/home screen.



Route Finding by Node ID

The first option we are given is to input our start and end nodes by using their numeric IDs, we can find these IDs by clicking on the node markers. Pressing find route will generate 3 separate routes after a short delay, these are given as a list of nodes along with their distance and a breakdown of indicator scores along the route. Colour coded routes are then drawn on the map. From these screenshots we can verify this stage of the test passes.

Route Finder

Start Node: End Node: Find Route Clear
Start Location: End Node: Find Route Clear Distance: Lighting: Greenery: Gradient: Crime:

Route from Node 300 to 600:

Red Path: 300 → 6 → 7 → 132 → 675 → 153 → 8 → 671 → 9 → 678 → 676 → 31 → 95 → 657 → 330 → 178 → 176 → 174 → 30 → 97 → 591 → 808 → 809 → 10 → 810 → 335 → 270 → 102 → 545 → 101 → 169 → 272 → 262 → 273 → 654 → 534 → 100 → 540 → 459 → 461 → 430 → 339 → 340 → 602 → 600

Total Distance: 2793.00

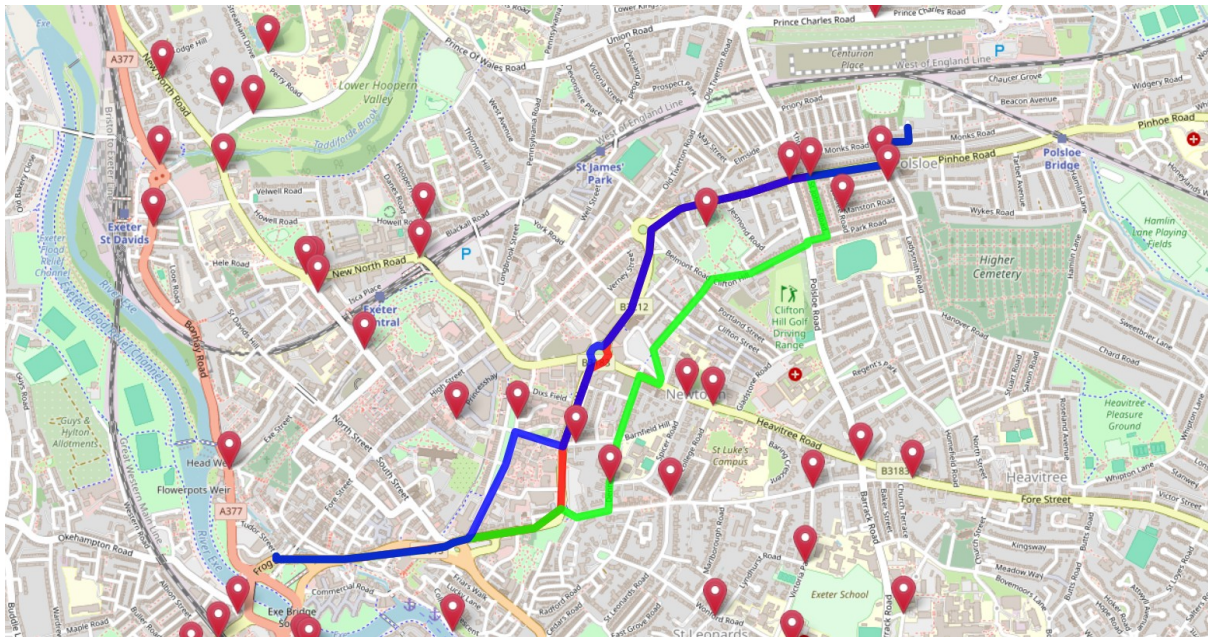
Score Breakdown: Lighting: 0.46Crime: 0.31Greenery: 0.48Gradient: 0.53

Green Path: 300 → 6 → 7 → 132 → 675 → 153 → 8 → 671 → 9 → 678 → 676 → 31 → 95 → 657 → 330 → 178 → 176 → 174 → 30 → 65 → 845 → 26 → 34 → 39 → 826 → 33 → 27 → 202 → 761 → 710 → 760 → 267 → 604 → 370 → 105 → 277 → 275 → 458 → 460 → 459 → 461 → 430 → 339 → 340 → 602 → 600

Total Distance: 2965.00

Score Breakdown: Lighting: 0.46Crime: 0.33Greenery: 0.46Gradient: 0.48

Blue Path: 300 → 6 → 7 → 132 → 675 → 153 → 8 → 671 → 9 → 678 → 676 → 31 → 95 → 648 → 99 → 679 → 327 → 96 → 321 → 97 → 591 →



Pass

Route Finding by Location Name

After clearing the map screen using the clear button. We go through the same process as above but enter a pair of named locations. These names can also be copied from clicking on the location nodes. This also works.



Pass

Route Finding with User Preferences

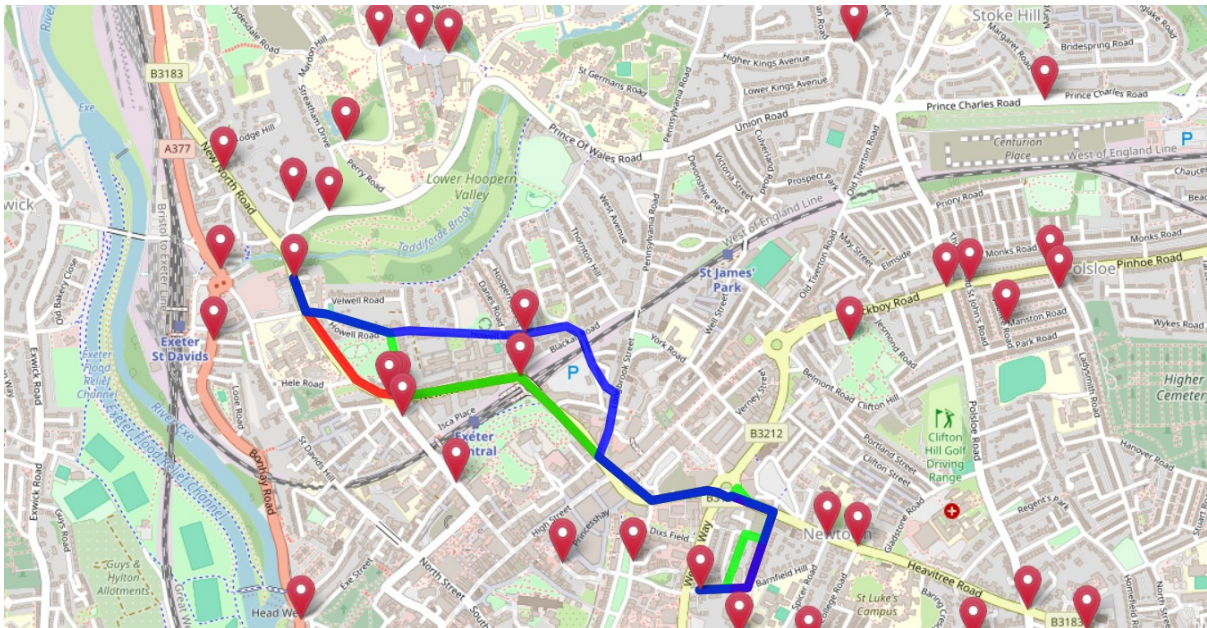
To test this we change the preference fields to some different values, here, prioritising Lighting and Greenery. Then we run the route-finding query again.

Route Finder

Start Node: 300 End Node: 600 Find Route Clear
 Start Location: The Imperial End Location: Southernhay Find Route Clear Distance: 0 Lighting: 1 Greenery: 1 Gradient: 0 Crime: 0

We can see that this generates different routes to the preferences we had before,

therefore the test passes and we know that our user-preference selection works with the weighted map.

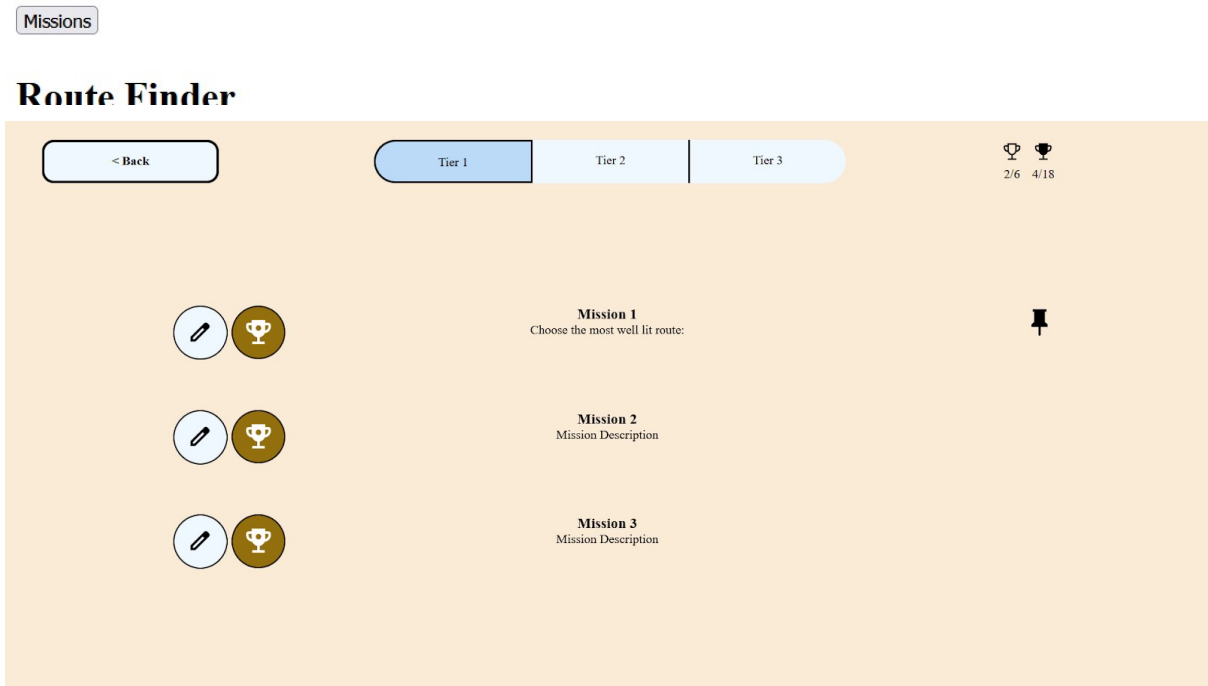


Pass

Mission Tests

Clicking on the mission button at the top of the page underneath the disclaimer will take us to the mission screen, as shown below.

Custom Routes in Exeter, UK
This tool does not reflect real world data
This tool is not suitable for emergency use
Crime and Lighting indicators are synthetic and do not reflect real world values



Selecting and Completing a Mission

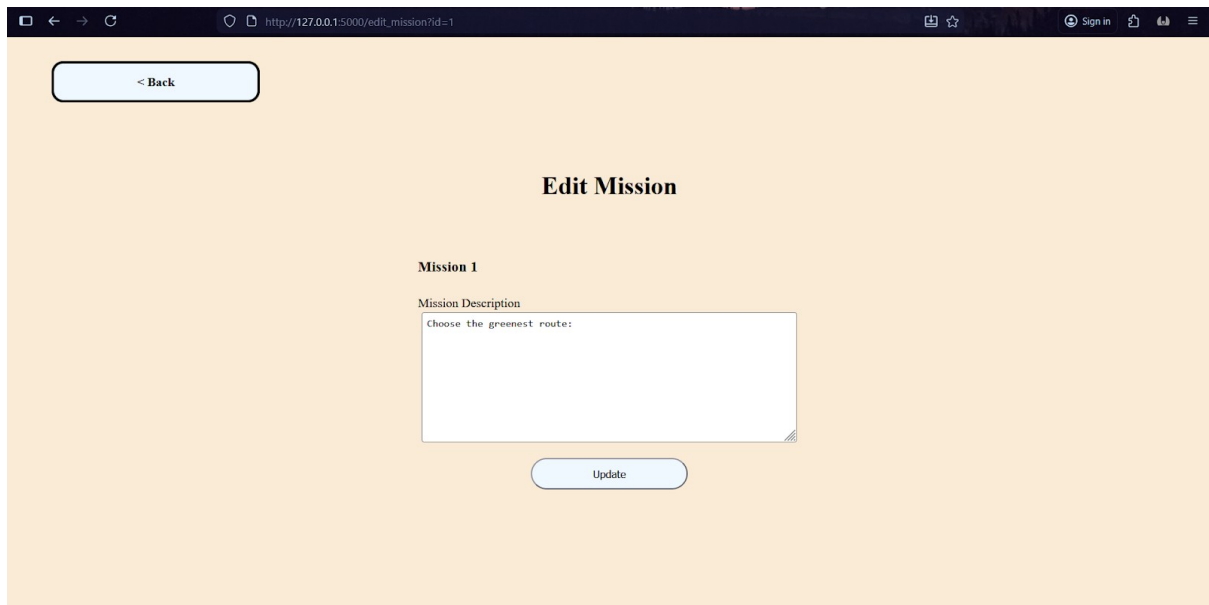
Next we click on Mission 1. Choosing either Blue or Green will show the user “incorrect”. Choosing red (the correct route) will give the user a “correct” pop-up and redirect them back to the mission screen.



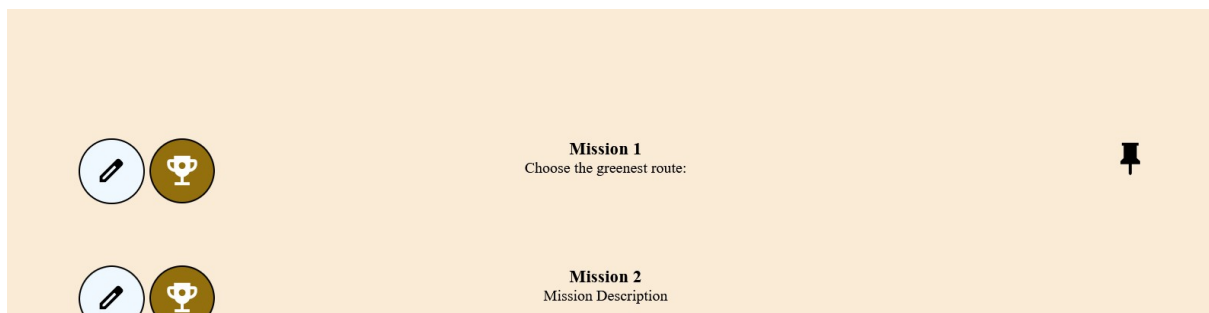
Pass

Editing Missions

Clicking on the pencil icon next to the mission will open the mission edit screen.



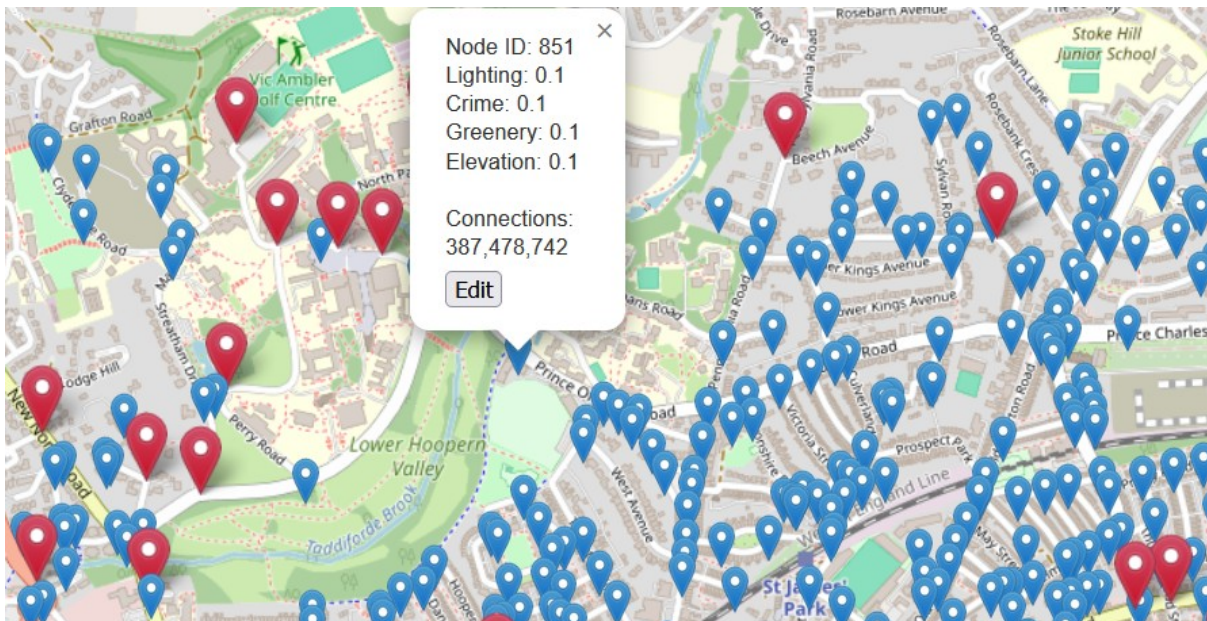
Change the text in the box and then click update. This change should now be reflected in the mission list.



Pass

Adding/Editing Nodes - Admin Features

Pressing the back button from the mission screen will take us back to the map screen. Here we are able to test some of the admin features. Left clicking on the map will create a new node, we can then add connections to the node. We can also edit the indicator values on the node. Restart the server and we can see the changes persist.



Right clicking on the map will open the Location Creation pop-up. This will also be saved on server reload.

